



Звіт про оригінальність

● Оцінка схожості

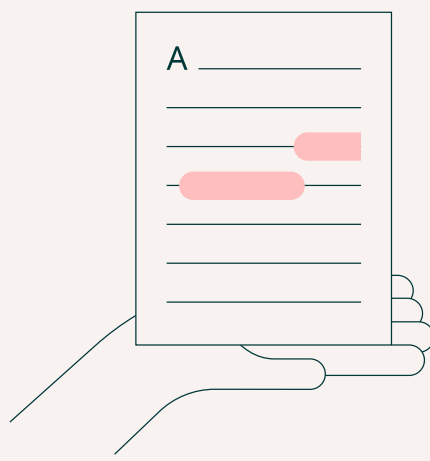
% 5

● Ризик плагіату

СЕРЕДНІЙ

👤 Offiks 🕒 2026-06-05 23:05

Посилання на звіт: 1aGee / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	5%
Перефразування	0%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	95%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.



5

Збіги

11 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ОДЕСЬКИЙ 1 НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА Факультет математики, фізики та інформаційних
технологій Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА з освітнього компоненту «Програмування»

на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ПРОДУКТОВИЙ МАГАЗИН».

Студентки 1 курсу, денна форма навчання

спеціальність F7 «Комп'ютерна інженерія»

Недєва Дарина В'ячеславівна

Керівник: к.т.н., доц. Антоненко О.С.

10 Національна шкала: _____

10 Кількість балів: _____

10 ECTS: _____

Дата захисту: _____

Члени комісії ____ Антоненко О.С.

1 (підпис) (прізвище та ініціали)

1 ____ 1 Шестопапов С.В.

1 (підпис) (прізвище та ініціали)

1 ____ 1 _____

1 (підпис) (прізвище та ініціали)

1 ОДЕСА - 2026

ЗМІСТ

ВСТУП3

ЗАВДАННЯ4

1. ТЕОРЕТИЧНА ЧАСТИНА5

1.1 Поняття ООП5

1.2 Опис об'єктної частини6

2. ПРАКТИЧНА ЧАСТИНА7

2.1 Опис класів7

2.3 Інтерфейс10

ВИСНОВКИ25

СПИСОК ЛІТЕРАТУРИ26

12 ВСТУП

12 Мета роботи – створити ієрархію класів на тему «Гіпермаркет» та її застосуванні для вирішення прикладної задачі, визначеної у варіанті завдання. У **8** ході виконання роботи необхідно реалізувати концепції об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно з вимогами завданням, слід розробити систему взаємопов'язаних класів, яка описуватиме структуру і функціонування продуктового магазину.

Для цього потрібно створити конструктори класів, визначити поля та реалізувати методи для роботи з об'єктами. Поля класів мають бути закритими, а доступ до них забезпечуватиметься через відповідні методи класу. У програмі необхідно побудувати ієрархію класів, базуючись на відносинах успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, класи, а за потреби - чисті віртуальні методи.

.

ЗАВДАННЯ

Варіант 20. Створення ієрархії класів на тему «Продуктовий магазин».

У роботі необхідно реалізувати класи, що описують структуру продуктового магазину,

товари та їх характеристики. Передбачити можливість роботи з різними категоріями харчових продуктів, зберігання інформації про їх назву, ціну, вагу, дата виробництва та інші характеристики. Також необхідно реалізувати взаємодію між класами, перевірку коректності даних та виконання основних операцій, пов'язаних із функціонуванням гіпермаркету.

1.ТЕОРЕТИЧНА ЧАСТИНА

Поняття ООП

Об'єктно-орієнтоване програмування – це концепсія, в межах якої програма розглядається як сукупність об'єктів, що обмінюються повідомленнями. Кожен об'єкт є екземпляром **3** певного класу, а класи, своєю чергою, утворюють ієрархію на основі наслідування. Розробник спочатку визначає класи (шаблони), а **9** під час виконання програми на їхній основі створюються конкретні об'єкти. Нові класи можуть успадковувати властивості від базових, що й породжує ієрархічну структуру.

Будь-який стиль програмування має свою концептуальну базу. Для об'єктно-орієнтованого стилю такою базою є об'єктна модель, яка ґрунтується на трьох основних принципах:

- а)інкапсуляція;
- б) поліморфізм;
- в) наслідування.

Інкапсуляція – механізм, який поєднує в одному об'єкті даних і методів їхньої обробки, а також приховування внутрішньої реалізації від зовнішнього світу, що запобігає помилковому чи несанкціонованому втручанням.

Наслідування – процес, завдяки якому один об'єкт (або клас) може набути властивостей іншого, доповнюючи їх власними, унікальними характеристиками. Це дозволяє будувати ієрархії класів від загального до конкретного.

Поліморфізм – здатність одного й того самого імені (наприклад, назви методу) позначати різні дії залежно від типу об'єкта, якому адресоване повідомлення. **7** На практиці це означає, що об'єкт сам обирає відповідний внутрішній метод відповідно до отриманих даних. Головна мета поліморфізму — забезпечити єдиний інтерфейс для класу споріднених операцій.

1.2 Опис об'єктної частини

Програма реалізує консольний застосунок для обліку харчових товарів на полиці

магазину. Система підтримує три категорії продуктів: йогурт, молоко та хліб. Кожен тип має власний набір характеристик, але всі вони успадковують спільні властивості від базового класу Product через проміжний клас ProductFood.

Магазин представлений полицею (Shelf), яка характеризується обмеженою місткістю (кількістю вільних місць для товарів). Полиця містить список екземплярів харчових продуктів. Новий товар може бути доданий лише в тому випадку, якщо на полиці є вільне місце; при спробі додати більшу кількість товарів, ніж дозволяє місткість, операція видає помилку.

Користувач взаємодіє із системою через консольне меню класу UserInterface, де може додавати товари обраного типу, переглядати асортимент на полиці, перевіряти термін придатності, сортувати товари за ціною, оформлювати покупку, переглядати виручку, перевіряти цілісність товарів та змінювати місткість полиці. При введенні некоректних даних (наприклад, від'ємна вага чи ціна, некоректні дати, перевищення допустимої жирності) програма виводить повідомлення про помилку за допомогою механізму винятків і продовжує роботу.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку 1. Розглянемо більш детально структуру кожного класу.

Базовий клас «Product»

Приватні поля: name, produceDate, weight, price.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

Методи доступу getName, getProduceDate, getWeight, getPrice – повертають значення відповідних полів;

Info – віртуальний метод, повертає текстовий опис товару(назва, дата виробництва, вага, ціна)

Клас «ProductFood», нащадок класу «Product»

Приватні поля: shelfLife, amountCalories.

Конструктори:

Конструктор без параметрів

Конструктор з параметрами

Методи:

Методи доступу getShelfLife, getAmountCalories – повертають термін придатності та кількості калорій;

Статичний метод convertDate – перетворює дати формату ДД/ММ/РРРР у числове значення;

Статичний метод dateComparison – метод перевірки, що дата виробництва не пізніша за термін придатності;

isShelfLifeGood — перевіряє, чи не прострочений товар відносно поточної дати;

Перевизначення Info — повертає розширений опис харчового продукту.

Клас «Yogurt», нащадок класу «ProductFood»

Приватні поля: fat, inside.

Конструктори:

Конструктор без параметрів

Конструктор з параметрами

Методи:

getFat, getInside — повертають жирність та начинку;

Перевизначення Info — повертає інформацію про йогурт.

4. Клас «Milk», нащадок класу «ProductFood»

Приватні поля: fat, packageType.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

getFat, getPackageType — повертають жирність молока та тип упаковки;

setFat — встановлює жирність з перевіркою допустимого діапазону (0–4 %);

Перевизначення Info — повертає інформацію про молоко.

Клас «Bread», нащадок класу «ProductFood»

Приватні поля: flourType, isWholeGrain.

Конструктори:

Конструктор без параметрів

Конструктор з параметрами

Методи:

getFlourType, getIsWholeGrain — повертають тип борошна та ознаку цільнозернового хліба;

Перевизначення Info — повертає інформацію про хліб.

Клас «Shelf»

Приватні поля: capacity, damage, amountMoney, vecAmountProduct.

Конструктори:

Конструктор без параметрів

Деструктор— звільняє пам'ять, виділену під об'єкти товарів.

Shelf::~Shelf()

Методи:

getCapacity, getAmountMoney, getDamageProduct — повертають місткість полиці, суму виручки та кількість видалених прострочених товарів;

sumPrice — додає ціну проданого товару до загальної виручки;

setCapacityIncrease, setCapacityReduce — збільшують або зменшують кількість вільних

місць на полиці;

addProductFood — додає товар на полицю;

loseShelfLifeProductFood — видаляє прострочені товари;

sortProductPriceAscending, sortProductPriceDescending — сортують товари за ціною;

buyProduct, buyAllProduct — оформлюють покупку одного товару або всіх товарів;

checkIntegrity, checkAllIntegrity — перевіряють цілісність товару за назвою або всіх товарів на полиці;

printProductFood — виводить перелік товарів на полиці.

Клас «UserInterface»

Методи:

SystemMenu — відображає головне консольне меню та обробляє вибір користувача (додавання йогурту, молока, хліба; перевірка терміну придатності; сортування; покупка; перегляд виручки; перевірка цілісності; виведення товарів; зміна місткості полиці; вихід із програми).

2.3 Інтерфейс

Під час запуску програми користувач бачить вітальне повідомлення та головне консольне меню.

Він потрапляє на це вікно одразу після запуску застосунку. На екрані відображається привітання WELCOME TO THE STORE, поточна дата магазину та перелік доступних пунктів меню (1–11). Для виконання дії потрібно ввести номер обраного пункту в рядку Your choice:. У програмі передбачено один тип користувача — оператор магазину, який керує полицею з товарами. (Рис.2.1).

Рисунок 2.1 – початкове (головне) меню програми

Після вибору пункту 1 програма пропонує ввести параметри йогурту: назву, дату виробництва (ДД/ММ/РРРР), вагу, ціну, термін придатності, калорійність, жирність (0–10 %), начинку, а також кількість одиниць товару. Перед додаванням виводиться кількість вільних місць на полиці. (Рис. 2.2)

Рисунок 2.2 – Вікно додавання йогурту.

Якщо запитана кількість перевищує вільні місця, з'являється повідомлення **4** There is

not enough space on the shelf, і товари не додаються. При коректних даних на полицю додається потрібна кількість йогуртів, після чого виводиться підтвердження. (Рис.2.3).

Рисунок 2.3 – Вікно при перевищенні вільних місць на полиці

Аналогічно пункту 1 користувач вводить назву, дати, вагу, ціну, термін придатності, калорії, жирність молока (0–4 %), тип упаковки та кількість(Рис.

При перевищенні місткості полиці операція скасовується з відповідним повідомленням(Рис. 2.3).

Рисунок 2.4 – Вікно додавання молока.

Користувач вводить назву, дати, вагу, ціну, термін придатності, калорії, тип борошна, ознаку цільозернового виробу (1 – так, 0 – ні) та кількість буханок(Рис. 2.5). Діють ті самі обмеження щодо вільних місць на полиці(Рис.2.3).

Рисунок 2.5 – Вікно додавання хліб

При введенні некоректних даних (наприклад, від’ємна вага(Рис.2.5) чи ціна(Рис.2.6), невірний формат дати(Рис.2.7), від’ємна калорійність(Рис.2.8), жирність поза допустимим діапазоном(Рис.2.8)) програма виводить текст помилки з винятку exception і повертає користувача до головного меню.

Рисунок 2.5 – Вікно при від’ємній вагі

Рисунок 2.6– Вікно при від’ємній ціні.

Рисунок 2.7 – Вікно при неправильному форматі дати

Рисунок 2.8 – Вікно при від’ємних калоріях

Рисунок 2.9 – Вікно при неправильному вводі жирності

Після вибору пункту 4 програма порівнює термін придатності кожного товару з поточною датою магазину. Прострочені товари видаляються з полиці, місце звільняється. На екран виводиться повідомлення про кількість знайдених прострочених позицій(Рис.2.11), або про те, що прострочених товарів немає(Рис.2.10).

Рисунок 2.10 – Результат перевірки терміну придатності

Рисунок 2.11 – Результат перевірки терміну придатності(коли термін придатності вийшов)

Користувачу пропонується обрати: (Рис.2.12).

сортування за зростанням ціни;

сортування за спаданням ціни;

вихід до головного меню.

Якщо на полиці немає товарів, програма повідомляє, що сортування неможливе(Рис.2.13).

Рисунок 2.12 – Підменю сортування за ціною

Рисунок 2.13– Підменю сортування за ціною(коли немає продуктів)

Доступні варіанти:

купити товар за назвою (потрібно ввести назву) (Рис.2.14);

купити всі товари на полиці(Рис.2.15);

повернення до головного меню(Рис.2.16);

При успішній покупці ціна товару додається до загальної виручки, товар зникає з полиці, місце звільняється(Рис.2.15). Якщо товар з такою назвою не знайдено, виводиться відповідне повідомлення(Рис. 2.17).

Рисунок 2.14 – Підменю оформлення покупки(за назвою)

Рисунок 2.15 – Підменю оформлення покупки всіх товарів

Рисунок 2.16 – Повернення до головного меню

Рисунок 2.17 – Результат покупки при не правильному вводиті назви

Після вибору пункту 7 на екран виводиться сума виручки в доларах(Рис.2.18) або повідомлення про те, що жоден товар ще не було продано(Рис.2.19);

Рисунок 2.18 – Виведення суми виручки

Рисунок 2.19 – Виведення того що жоден товар ще не було продано

Виведення підменю перевірки цілісності(Рис.2.20);

Користувач може:

перевірити цілісність товару за назвою(Рис. 2.21);

перевірити всі товари на полиці(Рис.2.22);

повернутися до головного меню(Рис.2.23)

Якщо товар знайдено, програма повідомляє, що він цілий(Рис.2.21); якщо назва не збігається з жодним товаром на полиці — виводиться повідомлення про відсутність товару(Рис.2.24).

Рисунок 2.20 – Підменю перевірки цілісності

Рисунок 2.21 – Підменю перевірених товарів на цілісність за назвою

Рисунок 2.22 – Підменю перевірених всіх товарів на цілісність

Рисунок 2.23 – Повернення до головного меню з перевірки на цілісність

Рисунок 2.24 – Випадок коли назва не збігається з жодним товаром на полиці

На екран виводиться повний перелік товарів із детальним описом кожного (метод Info()): назва, дати, вага, ціна, термін придатності, калорії та специфічні характеристики (жирність, начинка, тип упаковки, борошно тощо)(Рис.2.25). Якщо полиця порожня, виводиться відповідне повідомлення(Рис.2.25).

Рисунок 2.25– Список товарів на полиці

Рисунок 2.26– Вивід при порожній полиці

Відкриття підменю керування місткістю полиці(Рис.2.27).

Користувач може:

збільшити кількість місць на полиці(Рис.2.28);

зменшити кількість місць(Рис.2.29) (якщо введене значення більше за поточну місткість, операція відхиляється) (Рис.2.30);

переглянути поточну кількість вільних місць(Рис.2.31);

повернутися до головного меню(Рис.2.32).

За замовчуванням полиця має 6 вільних місць.

Рисунок 2.27 – Підменю керування місткістю полиці

Рисунок 2.28– Підменю керування місткістю полиці при збільшити кількість місць на полиці

Рисунок 2.29. – Підменю керування місткістю полиці при зменшити кількість місць на полиці

Рисунок 2.30 – Підменю керування місткістю полиці при зменшити кількість місць на полиці(при введені більшого числа за поточну місткість)

Рисунок 2.31 – Переглянути поточну кількість вільних місць

Рисунок 2.32 – Повернення до головного меню після керуванні полиці

Після вибору пункту 11 на екран виводиться прощальне повідомлення, і робота програми завершується(Рис.2.33).

Рисунок 2.33 – Вихід з програми

ВИСНОВКИ

5 Під час виконання курсової роботи я розробив програму на мові C++ на тему «Продуктовий магазин», застосовуючи підходи об'єктно-орієнтованого програмування. Було побудована ієрархія класів, що відображає структуру реального магазину. Кожен клас отримав власні поля та методи, а взаємодія між ними дозволяє моделювати основні процеси торгівлі.

При написанні програми активно використовувались принципи ООП. Інкапсуляція забезпечила захист даних усередині класів, наслідування дозволило побудувати логічну ієрархію від загального до конкретного, а поліморфізм — гнучко працювати з різними видами товарів через спільний інтерфейс. Програма підтримує декілька практичних сценаріїв: надходження нових товарів, їх розміщення по полицях, автоматичне списання прострочених продуктів та оформлення покупки. Також була додана можливість відстежувати виручку магазину — загальну суму коштів, отриманих від продажів.

Виконання цієї роботи допомогло краще зрозуміти, як працюють класи, колекції та механізми наслідування у C++, а також як **6** застосовувати їх для вирішення реальних задач. Розроблена програма показує, що навіть відносно проста система може повноцінно імітувати роботу справжнього продуктового магазину.

СПИСОК ЛІТЕРАТУРИ

С. А. Ярошко, Основи об'єктно-орієнтованого **2** програмування з ілюстраціями на

Borland Pascal та Borland Pascal for Windows [Електронне видання] / Львів 1998. – Режим доступу: <https://ami.lnu.edu.ua/wp-content/uploads/2020/04/OOP.pdf>

Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oop>

W3School, C# OOP (Object-Oriented Programming). Режим доступу:
https://www.w3schools.com/cs/cs_oop.php

Репозиторій GitHub на курсовий – Режим доступу:
<https://github.com/darynandedieva-coder/Kursach>

Додаток 1. Діаграма класів

Рисунок 1. – Діаграма класів

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	dspace.onu.edu.ua	1.1%
2	lnu.edu.ua	0.6%
3	careers.epam.ua	0.5%
4	context.reverso.net	0.4%
5	uchebe.net	0.3%
6	knuba.edu.ua	0.3%
7	iir.edu.ua	0.3%
8	ela.kpi.ua	0.3%
9	dspace.ksaeu.kherson.ua	0.3%
10	ru.essays.club	0.3%
11	irbis-nbuv.gov.ua	0.3%
12	styleshoesshop.ru	0.2%



Дякуємо, що перевірили
свій документ за допомогою
Plag!